

Axient: Manifest-Bound End-to-End Evidence for On-Chain Financial Protocols

Seven-Layer Derivation, Correlation, Tamper Rejection, and Reproducible Claim Promotion

Maksym Nechepurenko

ForesightFlow, Devnull FZCO, Dubai, United Arab Emirates

maksym@devnull.ae | [ORCID: 0000-0002-9515-8841](https://orcid.org/0000-0002-9515-8841)

August 2026

Abstract

Hybrid on-chain financial protocols are frequently evaluated with evidence that is individually useful but collectively insufficient: a unit test, transaction receipt, screenshot, service response, or several matching hashes may be presented as proof of a complete workflow even when the layers share one generated source or omit the financial authority that matters. This paper develops a manifest-bound evidence architecture for such systems. A sealed execution manifest binds the contract graph, roles, selectors, adapters, schemas, and release identity. Seven evidence layers are retained with registered derivations and linked by a common correlation identity. A conjunctive claim-promotion rule requires provenance, predicate-specific root conditions, negative controls, reproducible packaging, historical replay, and independent review in addition to hash equality. We formalize the evidence graph, prove that copied hashes cannot establish independent derivation, show how collision-resistant commitments make specified tampering detectable, define exactly-once financial-effect evidence, and distinguish original historical identities from later reproducibility attestations. The Phase 17B case study records twelve parent Financial Interaction Assertions in one accepted cohort: 84 retained `ACTUAL_EVIDENCE` positions, two clean materializations with one byte-identical file index, a deterministic strict archive accepted by a separate verifier, and a selected P01–P17 replay whose P17 output is content-bound to that archive. FIA-011 is conjunctive and retains both stale-before-admission and book-epoch-change-mid-position subcases. The result demonstrates a reviewable chain of custody from financial assertion to replay-bound package while preserving the difference between observed evidence, interpretation, and future hypotheses.

Keywords: end-to-end evidence; smart contracts; reproducibility; provenance; canonicalization; auditability; tamper detection; distributed systems; financial protocols.

1 Introduction

A complex financial protocol spans more than a blockchain. A user acts through an interface; an API authenticates and validates the command; a durable service orchestrates local or external execution; a smart contract changes financial state; indexers and journals reconstruct the result; and an audit surface presents a human-readable trace. The phrase “end-to-end” suggests that the same action has been exercised and observed across the full chain. Yet the evidence commonly used to support that claim is heterogeneous and easy to overstate.

A transaction receipt establishes that one transaction was mined. It does not establish that the intended user command produced it, that the called selector belonged to the approved deployment, that the service projection was derived from the receipt, or that the audit interface did not display fixture data. A screenshot establishes that pixels were rendered. It does not establish the state source. Seven equal hashes may still arise from one generator copied into seven files. A green CI run confirms

only the bounded task encoded in that workflow. These are not failures of the artifacts themselves; they are failures of claim typing.

Phase 17B of the Axient programme treats evidence as a first-class system. Its public evidence companion records one twelve-scenario cohort, 84 retained evidence positions, two matching clean materializations, a deterministic strict archive accepted by a separate verifier, and a selected P01–P17 replay bound to that archive [AxientLab, 2026]. The present paper generalizes the resulting architecture beyond event markets. Its object is a manifest-bound proof chain for hybrid financial protocols.

Contributions.

- (C1) A typed evidence model that distinguishes observed, reconstructed, replayed, and policy-level claims.
- (C2) A sealed execution manifest that binds code, configuration, roles, selectors, adapters, schemas, and release identity.
- (C3) A seven-layer evidence graph with separately retained inputs, registered derivations, and a shared correlation identity.
- (C4) A proof that hash equality alone cannot establish independent derivation.
- (C5) A claim-promotion predicate that is conjunctive rather than compensating.
- (C6) Tamper, drift, omission, duplication, and forbidden-authority controls.
- (C7) A reproducible retained-package specification with allowlists, manifests, SBOMs, clean runs, and independent review.
- (C8) A historical provenance model that distinguishes original release identities from later reproducibility attestations.
- (C9) An exactly-once recovery evidence object that proves one financial transaction across worker failure and projection rebuild.
- (C10) A twelve-parent FIA case study, including mandatory FIA-011A/B subcases, and a strict record of what remains not proven.

2 Problem statement and threat model

2.1 What an E2E claim means

Let q be a protocol claim, such as “a partial settlement reduced debt once and appeared consistently in the user and audit interfaces.” An end-to-end claim is not a single Boolean test. It is a statement about a chain of related observations under a specific system identity.

Let

$$\mathcal{L} = \{L_0, L_1, \dots, L_6\}$$

be evidence layers and let S be a sealed execution manifest. A claim q is supported only if each required layer has a registered derivation from retained inputs, all layers bind to the same manifest and correlation identity, the required derivation-root topology is satisfied, specified negative controls fail, and the complete package is reproducible and independently reviewable.

2.2 Adversarial evidence failures

The threat model includes both malicious and accidental failures:

- a result is detached from the commit or deployment that produced it;
- an API points to a stale address or selector;
- fixture expectations are copied into actual-state fields;
- multiple projections are generated from one sealed artifact and presented as independent runtime observations;
- evidence records or layers from distinct runs, manifests, or correlations are spliced into one synthetic cohort claim;
- event ordering is inferred from file order rather than chain coordinates;
- a journal entry is omitted, duplicated, or reordered;
- a browser displays cached or reference data;
- a package includes unallowlisted files, secrets, logs, or a self-referential checksum;
- a clean-run comparison excludes semantically relevant differences;
- a worker restarts after chain confirmation and financially re-executes the command;
- a historical phase is assigned a retroactive tag or expected hash that never existed;
- a green bounded test is promoted into a broader claim.

The goal is not to make all evidence impossible to forge in an absolute sense. The goal is to make the claim boundary explicit, machine-checkable, and difficult to broaden without producing new evidence.

3 Typed evidence and claim taxonomy

Definition 3.1 (Evidence item). *An evidence item is a tuple*

$$E = (\tau, s, d, h, c, m),$$

where τ is the evidence type, s the retained source identity, d the derivation description, h the digest, c the correlation identity, and m the manifest identity.

The primary evidence types are:

- **FORMAL**: theorem under stated assumptions;
- **DETERMINISTIC_BENCHMARK**: exact registered cases;
- **OBSERVED_LOCAL**: chain receipt, state, or process fact observed locally;
- **RECONSTRUCTED_PROJECTION**: materialized service/API/browser/audit state;
- **MECHANICAL_REPLAY**: result of a frozen workflow replay;
- **POLICY**: owner or governance design choice;
- **NOT_PROVEN**: required layer or gate is absent.

Proposition 3.2 (Non-substitutability of evidence types). *No number of reconstructed projections can substitute for an absent observed canonical transaction or state transition, and no number of unit tests can substitute for an absent historical replay when the claim includes replayability.*

Proof. The evidence types assert different predicates. A projection asserts consistency with its source, not occurrence of a canonical financial transition. A unit test asserts a property of the test environment, not reproducibility of a historical workflow. Summing evidence of one type cannot establish a logically distinct missing predicate. $\square$

4 Sealed execution manifest

The manifest anchors what system was exercised. It should contain:

- (i) source commit or tag and repository identity;
- (ii) chain ID and execution mode;
- (iii) contract addresses, creation/deployed bytecode, ABIs, and storage layouts;
- (iv) roles, owners, selector policy, and pause target set;
- (v) adapter and venue-emulator identity;
- (vi) OpenAPI, schema, generated-client, and UI build identities;
- (vii) dependency lock and toolchain identities;
- (viii) manifest digest.

Let S be the RFC 8785 canonical JSON representation [Rundgren et al., 2020]. Its digest is

$$h_M = H(S).$$

Definition 4.1 (Manifest-bound evidence). *An evidence item is manifest-bound if its canonical representation includes h_M and the verifier rejects any mismatch between h_M and the runtime identities used to produce the item.*

Proposition 4.2 (Drift visibility). *If address, bytecode, ABI, role, selector, adapter, schema, or client identity is included in S , any change to those fields changes h_M under collision resistance. A result under the changed system cannot retain the old manifest identity without a hash collision or a verifier defect.*

The manifest is analogous to supply-chain provenance: it binds the artifact to the source and process that produced it [Torres-Arias et al., 2019, Open Source Security Foundation, 2023, Souppaya et al., 2022]. Unlike a generic build attestation, it also binds financial authority and permitted command surfaces.

5 Seven retained evidence layers and registered derivations

The evidence model uses the layers in table 1, and their manifest/correlation linkage is illustrated in fig. 1. The word *retained* is deliberate: a layer can be useful without being an independent root.

For layer i , let I_i be the retained input set, D_i the derivation, and

$$h_i = H(\text{JCS}(D_i(I_i, h_M, c))).$$

Table 1: Seven retained evidence layers.

Layer	Source	Required property
1. Simulator expectation	registered vector/fixture	expectation only; never financial authority
2. Canonical contract state	manifest addresses, ABI views, ordered events	observed from the deployed graph
3. Append-only journal	transaction/event identifiers	complete, ordered, no silent overwrite
4. Service projection	journal replay	registered materialization, non-authoritative
5. Authenticated API	service projection under role/session	registered derivation, route, identity, response, no bypass
6. Browser projection	API/service state after a real role control or typed browser client	visible state tied to command/correlation; not presumed independent
7. Audit reconstruction	correlation and journal replay	reviewer-readable, separately rebuilt where policy requires

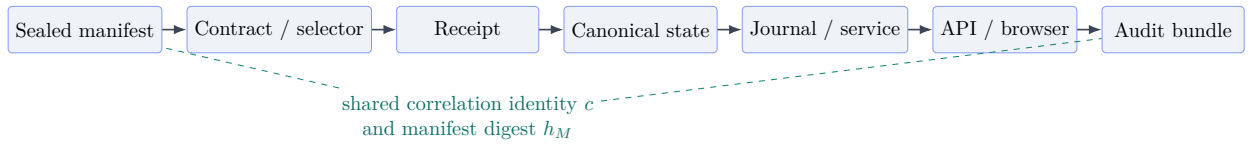


Figure 1: Manifest-bound retained evidence chain. The diagram is sequential for readability; verification also checks cross-links, provenance, and registered derivations. A proposed multi-root corroboration gate would be required for a stronger independence claim, but was not evaluated here.

Definition 5.1 (Registered derivation). *A retained layer record has a registered derivation if it identifies its input set, transformation, root ancestry, implementation/runtime identity, output digest, and correlation/manifest bindings. Registration does not by itself imply independence.*

Definition 5.2 (Multi-root corroboration). *For claim q , let $R(q)$ be the set of retained root observations in the evidence graph. The claim satisfies multi-root corroboration if the policy requires it and no single generated artifact lies in every derivation cut for q ; equivalently, at least two declared root paths can independently falsify the claimed semantics under the registered failure model.*

Theorem 5.3 (Copied-hash insufficiency). *Suppose $h_1 = \dots = h_7 = h$, but every layer was generated by copying one fixture value h . Then equality provides no evidence that any actual layer observed or reconstructed the claimed state.*

Proof. The equality is entailed by the copying procedure alone and is independent of the system execution. Therefore it has zero discriminating power between a correct execution, an incorrect execution, and no execution. Separately retained inputs with registered derivations are necessary to connect equality to the protocol; claims of independent runtime participation would additionally require a proposed multi-root corroboration gate. That gate was not evaluated for Phase 17B and remains NOT_PROVEN. $\square$

6 Correlation identity

A scenario must be traceable across user, service, chain, and audit layers. Define

$$c = H(\text{JCS}(\text{scenario}, h_M, \text{role}, \text{command}, \text{idempotency domain})).$$

The retained correlation ledger should include:

- UI route, control ID, role/session ID;

- API endpoint, command ID, payload and idempotency-key digests;
- saga, inbox, and outbox IDs;
- emulator order/match/settlement IDs;
- settlement-evidence ID;
- contract address, selector, calldata digest, transaction hash;
- ordered event identifiers;
- journal entries and projection identities;
- browser and audit reconstruction identities.

Proposition 6.1 (Correlation exclusion). *If the verifier rejects events, addresses, selectors, or projections that are not manifest-authorized or not bound to c , a legacy bridge or off-path action cannot silently satisfy the canonical claim.*

A correlation ledger is not itself financial authority. Its purpose is to make the exact authority path inspectable and to fail when a required observation is missing.

Definition 6.2 (Cross-cohort splice). *A cross-cohort splice is a constructed evidence package in which at least one load-bearing record or layer is drawn from a different run, manifest, scenario correlation, or parent-record identity while the package is presented as one coherent cohort.*

Definition 6.3 (No-splice gate). *For claim q , the no-splice gate passes only if every required record and derivation edge resolves to the same declared cohort identity, each scenario correlation occurs exactly once, and no layer is admitted through an identity not reachable from that cohort’s manifest and ledger root.*

Proposition 6.4 (No-splice soundness under complete identity validation). *Assume collision resistance and complete validation of run, manifest, correlation, scenario-record, and layer-root identities. Then replacing a required layer with an object from another cohort causes the no-splice gate to fail unless a checked identity is also forged.*

Proof. A foreign object differs in at least one declared run, manifest, correlation, scenario record, or root ancestry. Complete validation checks these fields against the cohort registry. The substitution therefore changes a checked digest or violates an identity predicate. $\square$

7 Evidence graph and independence model

7.1 Directed evidence graph

As illustrated in fig. 2, represent an evidence package as a directed acyclic graph

$$G_E = (V_E, E_E),$$

where each vertex is an evidence item and each directed edge records a derivation or reference relation. Root vertices are retained observations or registered expectations; non-root vertices are projections, normalizations, package manifests, or review decisions.

A valid graph must satisfy:

- (E1) every non-root vertex names all load-bearing parents;
- (E2) every edge is typed, for example observed-from, derived-from, replayed-from, or reviewed-against;

- (E3) every vertex binds to the same scenario correlation identity and to an approved manifest identity;
- (E4) cycles are prohibited unless the cycle is only a checksum self-reference explicitly excluded from the package identity;
- (E5) a layer claimed to have an independent runtime root cannot have the compared layer's normalized output as its only parent.

Definition 7.1 (Derivation cut). *A derivation cut for claim q is a set of root observations whose removal makes the evidence graph unable to derive q . A claim has multiple independent roots only if no single generated artifact lies in every derivation cut.*

This definition provides a practical test for pseudo-independence. If all five application projections are materialized from one sealed JSON object, that object lies in every derivation cut. Domain-separated hashes prevent accidental collisions but do not create observational independence.

Proposition 7.2 (Common-root limitation). *If every actual-layer record is a deterministic function of one generated artifact z and no independent observation of the claimed financial state is retained, then the evidence proves consistency with z , not independent participation of the corresponding runtime layers.*

Proof. All evidence paths factor through z . Replacing the real system with any generator that emits the same z leaves every derived record unchanged. Therefore the package cannot discriminate real participation from generated consistency. $\square$

7.2 Independence classes

The framework distinguishes four useful independence classes:

- **syntactic independence**: different files, schemas, or domain tags;
- **implementation independence**: different code paths or languages;
- **runtime independence**: separately running components observe or reconstruct the event;
- **operator independence**: a distinct reviewer or process validates the retained package.

None implies the next. The final claim matrix should state which class is achieved for each layer.

Proposition 7.3 (Disguised-layer rejection). *If an $L4$ service object is serialized under an $L5$ label without an authenticated API observation and distinct registered derivation, the evidence graph preserves the $L4$ root ancestry and the $L5$ gate fails. This negative control is proposed as *NC-ROOT-001*; it was not evaluated in the Phase 17B reference cohort.*

7.3 Evidence completeness and minimality

An evidence package should be complete enough to reproduce the declared claim but minimal enough to avoid unnecessary secrets, logs, and personal data. Let F be the allowlisted file set and q the claim. The package is *claim-complete* if an independent verifier can evaluate every gate in $P(q)$ using F and the declared external toolchain. It is *minimal* if removing any load-bearing file makes at least one required gate unevaluable.

The minimality criterion is not a demand for the smallest byte size. It is a check against packaging arbitrary caches, videos, process dumps, or private directories that do not contribute to verification.

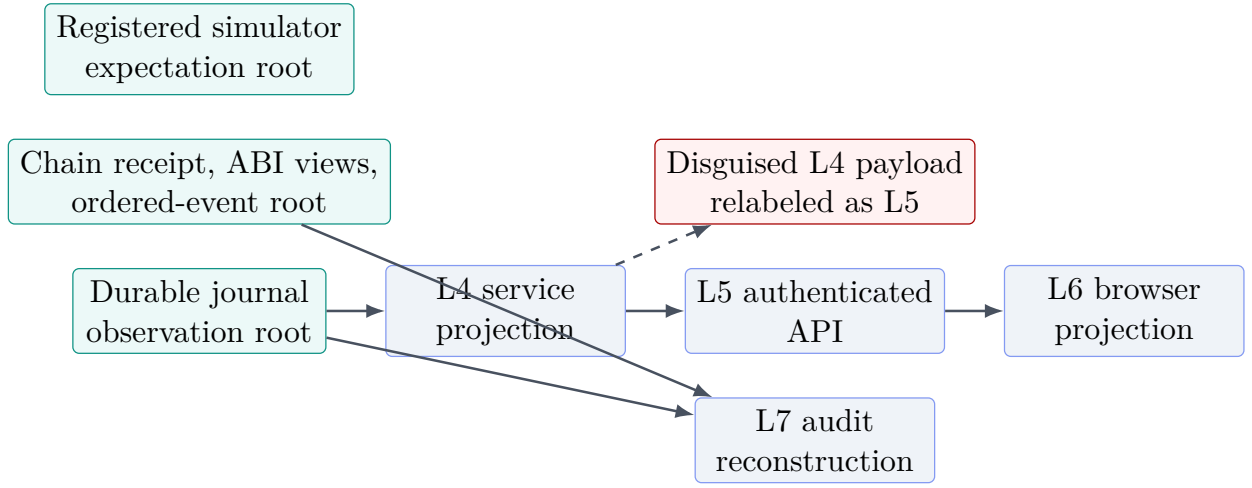


Figure 2: Registered derivation-root topology. Service, API, and browser layers may share one projection root. The framework therefore reports root ancestry and rejects an L4 object merely repackaged as L5. Multi-root corroboration is imposed only for claims that require independent participation.

8 Predicate classes and claim promotion

Not every promotion predicate is monotone under appended evidence. The proposed framework classifies each gate as one of:

- **MONOTONE_POSITIVE**: once true under append-only valid evidence, it remains true;
- **TERMINAL_CLOSURE**: evaluated only when the run or package is sealed;
- **ABSENCE_NEGATIVE_CONTROL**: absence of a prohibited event over a bounded terminal domain;
- **RECOMPUTED_COMPARISON**: a clean-run, replay, or package comparison recomputed from current inputs;
- **MULTI_ROOT_CORROBORATION**: a proposed derivation-topology predicate recomputed when roots or edges change; it was not evaluated for Phase 17B.

Blanket upward monotonicity is invalid for the last four classes. Package closure can be broken by an extra member, an absence gate can fail when a prohibited event arrives, and a comparison or root topology can change when evidence is appended.

9 Claim promotion as a conjunctive predicate

Let the required gates for claim q be $g_1(q), \dots, g_n(q) \in \{0, 1\}$. Define

$$P(q) = \bigwedge_{k=1}^n g_k(q).$$

The claim is promoted only if $P(q) = 1$.

For a full local E2E claim, representative gates include:

- (G1) sealed manifest and approved command path;
- (G2) canonical receipt or typed no-mutation rejection;
- (G3) canonical state observation;

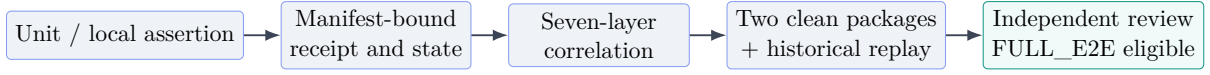


Figure 3: Claim-promotion ladder. Every stage adds a qualitatively distinct predicate; later language must not be applied early.

- (G4) seven retained derivations;
- (G5) shared correlation identity;
- (G6) no-splice cohort-identity gate;
- (G7) mandatory negative controls;
- (G8) two clean complete packages;
- (G9) full historical replay;
- (G10) independent final-package review;
- (G11) no forbidden bridge, external write, production asset, or leverage above the scoped limit.

Theorem 9.1 (No compensating promotion). *Under the conjunctive predicate, passing additional non-critical tests cannot compensate for a failed required safety or provenance gate.*

Proof. A conjunction is false when any conjunct is false, independently of the number of true conjuncts. Therefore performance, code coverage, or extra passing cases cannot hide a missing required layer. $\square$

This rule differs from a weighted maturity score. A score may be useful for project management, but it is unsuitable for a claim whose meaning includes every specified layer. The resulting promotion sequence is shown in fig. 3.

10 Tamper and drift controls

10.1 Mutation families

The verifier should intentionally mutate:

- manifest, ABI, bytecode, storage, address, role, or selector identity;
- receipt status, log digest, event order, or transaction index;
- fixture hash propagated into an actual layer;
- omitted, duplicated, or reordered journal entries;
- stale service/API/browser projections;
- an L4 service payload disguised as an L5 authenticated-API observation (proposed control NC-ROOT-001, not evaluated in Phase 17B);
- a scenario record or layer spliced from a different run, manifest, correlation, or cohort root;
- correlation IDs and idempotency domains;
- output-root, ZIP members, checksum files, and allowlists;
- external-write markers, non-loopback routes, production identifiers, or over-2 $\times$ settings;

- duplicate and conflicting settlement evidence;
- pause target failure, premature unpause, or recovery reexecution.

Assumption 10.1 (Collision resistance and complete validation). *The hash function is collision resistant for the relevant inputs, and the verifier checks every field declared load-bearing by the schema and policy.*

Theorem 10.2 (Specified-tamper detection). *Under collision resistance and complete validation, any mutation to a load-bearing retained field either changes a checked digest or violates a schema/policy predicate, and is therefore rejected.*

The theorem is intentionally scoped. A field omitted from the schema or not checked by policy can be mutated without violating the theorem. Mutation testing is therefore a method for discovering missing load-bearing fields [Jia and Harman, 2011].

10.2 Differential and property-based validation

Independent implementations should canonicalize and verify the same objects. Differential testing catches implementation-specific mistakes [McKeeman, 1998]; property-based testing exercises invariants across generated inputs [Claessen and Hughes, 2000]; smart-contract fuzzers and multi-tool comparisons provide complementary coverage [Wüstholz and Christakis, 2020, Durieux et al., 2020]. None of these methods alone proves a claim, but together they strengthen the evidence package.

11 Retained evidence package

A portable evidence package must be reviewable without exposing secrets or relying on temporary paths. The allowlisted contents may include:

- manifest, schemas, source/release identity, and toolchain metadata;
- per-scenario command, receipt, state, event, journal, service, API, browser, audit, reconciliation, and validator records;
- negative-control results;
- two clean-run indexes and comparison;
- replay attestation;
- SBOM and dependency/license inventory;
- privacy/secret scan and independent review report.

Excluded by default are credentials, keys, cookies, raw logs, videos, dumps, private RPCs, personal data, raw third-party venue data, historical harness evidence presented as canonical, and self-referential checksum entries.

Definition 11.1 (Package identity). *For an ordered allowlisted member set $F = (f_1, \dots, f_k)$, the package identity is the digest of the deterministic archive construction and a manifest containing every member digest, size, mode, and canonical path.*

Two clean packages may be required to be byte-identical. When raw runtime fields are inherently variable, the policy must separate raw identities from a normalized semantic identity and justify each excluded field. A normalized comparison cannot be silently described as byte-identical.

12 Historical replay provenance

Long programmes often lack a frozen replay wrapper or aggregate output hash for an early phase. Backdating a tag or assigning a historical expected hash creates fictitious evidence.

Definition 12.1 (Post-hoc reproducibility attestation). *A post-hoc reproducibility attestation binds an already recorded source boundary, a later-created replay wrapper, the pinned environment reconstructed from historical lockfiles, and the newly observed replay result. It is not the original historical release identity.*

For phase p , record

$$B_p = (\text{boundary type, commit/tag, historical replay status, new attestation class}).$$

If no historical aggregate hash existed, the field remains null with a reason. This discipline preserves provenance while still permitting useful later replay.

Proposition 12.2 (No retroactive identity). *A post-hoc replay hash cannot be represented as an original historical expected-output hash without making a false statement about what was frozen at the earlier time.*

13 Exactly-once recovery evidence

A financial recovery claim must prove more than application availability. For command x , retain:

- (1) chain-confirmed transaction and receipt;
- (2) durable checkpoint written before process termination;
- (3) old process identity and termination evidence;
- (4) fresh process identity;
- (5) inbox/outbox replay record;
- (6) registry replay/no-op classification;
- (7) exactly one financial transaction in the canonical domain;
- (8) rebuilt journal, service, API, browser, and audit projections.

Definition 13.1 (Exactly-once evidence object). *An exactly-once evidence object is the tuple*

$$E_{xo} = (x, h_M, c, \tau, C, p_{old}, p_{new}, R, \Delta_f, \Pi),$$

where τ is the canonical transaction, C the durable checkpoint, R the replay result, Δ_f the duplicate financial delta, and Π the rebuilt projection set.

The acceptance condition is

$$\#\{\text{canonical financial transactions for } x\} = 1, \quad \Delta_f = 0.$$

Exactly-once delivery is not claimed; exactly-once financial effect under the specified recovery protocol is.

14 FIA case study

The twelve FIA cases cover the evidence questions summarized in table 2:

Table 2: FIA evidence questions.

FIA	Financial scenario	Evidence question
01	successful admission	can UI/API/origination receipts and canonical state be correlated?
02	exact 2× admission	is the leverage policy and calldata identity preserved?
03	risk rejection	is typed rejection proven with zero mutation?
04	partial then final settlement	do distinct admitted receipts bind bounded partial application and final settlement in one lifecycle record?
05	residual and collateral release	are trader residual and collateral released once after canonical debt-clearance eligibility?
06	liquidation to hard-flat	are authorized liquidation request/execution and the hard-flat transition bound to one correlation?
07	waterfall	do reserve, LBP, and Senior state changes conserve ordered loss?
08	conflict	is quarantine observed with zero debt mutation?
09	Senior withdrawal queue	do queued CreditPool shares remain loss-participating until service?
10	recovery	is one financial transaction preserved across process restart?
11	conjunctive capability transition	do FIA-011A stale-before-admission and FIA-011B book-epoch change with $D_t > 0$ each pass retained-evidence, causal, fail-closed/no-mutation, negative-control, replay, and report gates?
12	pause/recovery	are target set, guardian pause, timelock, and unpause ordered correctly?

The cases are not an aggregate score. A missing evidence layer in FIA-10 cannot be offset by extra tests in FIA-01. FIA-011 is one parent assertion and passes only when both FIA-011A and FIA-011B close with retained raw and normalized evidence, manifest-bound identity, exact transition provenance, causal checks, expected no-mutation or fail-closed semantics, negative controls, replay, and explicit per-subcase report coverage. One passing subcase cannot close the parent, which remains blocked while either evidence path is incomplete. The same principle motivates the AEMB v0.2 extension developed in the companion benchmark article.

15 Observed Phase 17B result

Observed Phase 17B result. Twelve uniquely correlated FIA records were retained under one manifest/root with 84 **ACTUAL_EVIDENCE** positions, two matching clean materializations, a deterministic strict archive accepted by a separate verifier, and a selected P01–P17 replay bound to the archive.

The accepted all-12 run is `phase17b-run-4c40d373b9c4ead91ecfdcab5539941743733b06`, produced from workspace commit `bd6b2d54ec6e897b49756399637aaa49647b8ce1`. The cohort manifest is `342b2a6f27aeb91bba2013f5604ab1c50aaba48d6f218c3f3f84cc5c39ce7bfd`, and the ledger root is `09cd709444eb82a0c2c17d4b42a2750b2cd9b6002f69b6a3648c4e2ea1b41e46`. Each parent FIA occurs exactly once and retains seven **ACTUAL_EVIDENCE** positions, for a total of 84.

Table 3: Observed evidence-chain milestones.

Object	Observed identity/result
All-12 manifest	342b2a6f...39ce7bfd; 12 records, 84 positions
Clean materializations	A 8d0fc26b...b626b079; B b69d6073...89065dfd
Shared file index	16 files, 122,636 bytes; 5f1f78f1...7f5661fc
Comparison	byte-identical at registered boundary; 53ebd95f...b3447187
Strict archive	16 members, 88,569 bytes; 6901e327...e54cf328
Replay	17 callbacks; coordinator 2e51296a...18bb0767f
P17 binding	output 111a9c6c...fa6cda1b; binding 7363650e...3babba64

The completed Phase 17B identities are summarized in table 3.

The two clean trees were produced through separate source reads and have act-specific manifests but one canonical file-index identity. The strict packager produced fixed archive content, and a separate verifier reopened the archive bytes. The selected replay preserves versioned causal edges such as P04–P05-v3, P06–P07-v3, and P15–P16-v2 before binding P17 to the strict archive.

FIA-011 provides a useful evidence-contract example. Its parent record contains two mandatory retained subcases: stale capability before admission and a later book-epoch change after a position has opened. The source-owned adapter and focused tests require both members; FIA-03 or a one-receipt placeholder cannot satisfy the parent.

15.1 Observed, interpreted, and future claims

Observed: the identities, counts, byte comparison, verifier acceptance, and replay binding above. **Interpretation:** the evidence architecture preserves a chain of custody from financial assertion through projections, deterministic materialization, packaging, and replay. **Future hypotheses:** external venues, public networks, real custody, terminal settlement, and production operations require new evidence contracts rather than reinterpretation of this cohort.

16 Evidence package schemas

16.1 Scenario evidence record

A per-scenario evidence record should contain the following groups:

- (a) **identity:** schema, scenario, manifest, correlation, source release;
- (b) **command:** role, session class, route, endpoint, command, idempotency domain, payload digest;
- (c) **chain:** target, selector, calldata digest, transaction, receipt, ordered events;
- (d) **state:** terminal block, ABI-view inputs, canonical state projection and digest;
- (e) **application:** journal, service, API, browser, and audit inputs and digests;
- (f) **negative controls:** mutation identifiers, expected rejection, observed rejection;
- (g) **recovery:** checkpoint/process/replay fields when applicable;
- (h) **verdict:** gate vector, missing layers, classification, reviewer notes.

A simplified record is:

```
{
  "schema": "axient/e2e-evidence/v1",
```

```
"scenario_id": "FIA-010",
"manifest_hash": "...",
"correlation_id": "...",
"command": {...},
"chain": {...},
"canonical_state": {...},
"journal": {...},
"service": {...},
"api": {...},
"browser": {...},
"audit": {...},
"recovery": {...},
"negative_controls": [...],
"verdict": {...}
}
```

16.2 Release index

The release index should record every scenario, file, digest, test, replay, and review artifact. It also records deliberate absences. A field such as `historical_expected_output_hash=null` is preferable to inventing a value; the reason field explains that the hash was not frozen at the historical phase.

16.3 Allowlist and path safety

The package builder must reject absolute paths, parent-directory traversal, symlinks where prohibited, duplicate ZIP members, executable binaries not declared by policy, and files outside the allowlist. Checksum files exclude their own line to avoid circular identity. Build outputs should be generated in clean OS-temporary or designated durable evidence roots, never collected by recursively archiving an active developer worktree.

17 Independent review protocol

An independent review has two stages.

17.1 Mechanical verification

The reviewer verifies:

- (M1) outer and inner archive hashes;
- (M2) member paths, sizes, modes, and manifest entries;
- (M3) schemas and canonicalization;
- (M4) source, build, toolchain, and manifest identities;
- (M5) every gate and negative control;
- (M6) two clean-run comparison;
- (M7) historical replay provenance;
- (M8) secret, privacy, data-rights, and license boundaries.

17.2 Semantic verification

The reviewer then asks whether the retained artifacts actually support the language used. Examples:

- Does a chain receipt represent the intended selector or merely a harness event?
- Was the browser state caused by a role control or copied from a fixture?
- Does a recovery report show a fresh process and one financial transaction?
- Do five projections have separately observed runtime roots under the registered derivation graph, or are they five views from one generator?
- Does the declared FULL_E2E scope include the relevant contracts and services?

Definition 17.1 (Evidence-review outcome). *The review outcome is one of the following:*

PASS PASS_WITH_LIMITATIONS FAIL_CLOSED.

Each outcome is accompanied by a machine-readable finding registry. The outcome does not change the evidence; it constrains the claims that may be made from it.

18 Detailed tamper matrix

The principal mutation targets and required detections are listed in table 4.

Table 4: Representative tamper controls.

Target	Mutation	Required detection
Manifest	change address, ABI, bytecode, role, selector, adapter, schema	manifest verifier rejects before execution or materialization
Receipt	flip status, tx digest, topic, data, chain ID	receipt/state verifier rejects
Event order	swap block/transaction/log ordering	canonical state digest changes or ordering gate rejects
State	copy fixture expected hash into actual state	source-provenance gate rejects
Journal	omit or duplicate canonical event reference	journal completeness/replay differs
Service	materialize stale or off-manifest state	service projection/manifest gate rejects
API	alter role/session or response field	authenticated response and hash mismatch
Browser	show cached/reference result	browser/source/correlation proof fails
Audit	omit command or selector edge	reconstruction completeness fails
Cohort	splice record or layer from another	no-splice identity gate fails
Package	run/manifest/correlation inject member, change path, remove file	allowlist/manifest/checksum fails
Recovery	create a second financial tx	exactly-once gate fails

Table 5: Evidence-to-language examples.

Evidence	Allowed language	Prohibited promotion
unit/local test	“the local test passed”	“the protocol is E2E validated”
manifest-bound receipt	“a canonical local receipt was observed”	“the full user workflow completed”
seven retained layers	“cross-layer evidence with registered derivations was retained”	“independent systems all proved the result” unless a separately evaluated multi-root gate is shown
two clean packages	“the package rebuilt reproducibly under the stated policy”	“production is deterministic”
full replay + independent review	“the scoped local E2E claim passed”	“production-safe, audited, or live-venue validated”

Target	Mutation	Required detection
Boundary	include remote RPC, external write, real asset, over-limit leverage	environment policy fails

19 Claim-language governance

Technical evidence should map to approved language. Representative mappings are given in table 5.

This governance is not cosmetic. It prevents a technically correct narrow result from becoming a misleading public claim.

A Formal properties of evidence promotion

A.1 Predicate-class-aware monotonicity

If the required gate set changes from G to $G' \supseteq G$, a claim that passed under G may fail under G' . This is not evidence regression; it is a stricter claim definition. Historical results retain their original scope and are not retroactively relabeled.

Proposition A.1 (Monotonicity of positive append-only gates). *Fix a manifest and scope. Let g be classified `MONOTONE_POSITIVE`, and suppose its truth depends only on the presence and validity of an append-only witness set. Appending additional valid, non-contradictory evidence cannot change $g = 1$ to $g = 0$.*

Proof. By definition, the witness establishing $g = 1$ remains present and valid after append-only extension. Because the predicate does not assert closure, absence, comparison equality, or root minimality, the extension does not invalidate the witness. $\square$

Remark A.2 (Non-monotone gates). *The classes `TERMINAL_CLOSURE`, `ABSENCE_NEGATIVE_CONTROL`, `RECOMPUTED_COMPARISON`, and `MULTI_ROOT_CORROBORATION` are terminal or recomputed predicates. They must be evaluated again whenever the bounded evidence set or derivation topology changes.*

A.2 Hash soundness versus semantic completeness

Collision resistance supports byte-level tamper detection, but semantic completeness depends on the schema. If a load-bearing field is omitted from canonicalization, the hash can remain stable while the system meaning changes. Mutation testing and independent review are therefore part of the evidence model, not optional QA.

A.3 Independence is relational

No layer is independent in isolation. Independence is a relation between derivation paths and failure modes. Two languages can share one faulty specification; two processes can share one stale database; two reviewers can use one verifier. The paper therefore reports independence classes rather than a binary adjective.

B Public evidence archive

The public companion at <https://github.com/AxientLab/axient-fia-public> retains sanitized manifests, FIA definitions, schemas, provenance records, checksums, and an offline publication verifier. The pinned companion commit is 7722146bee0e7f02d0c4f9f63ada6d527e6ff079. The Phase 17B seven named layers are registered derivations whose publication records share the accepted cohort metadata; the companion does not publish the raw runtime payloads needed to infer seven independent observations. The verifier recomputes repository-file checksums and validates schemas, counts, ordering, canonical identifiers, and internal consistency; it cannot reconstruct the deliberately unpublished raw roots or independently recompute their payload-derived manifest, ledger, archive, or replay digests. Run it with `node verifier/verify.mjs`.

C Relation to assurance and distributed-systems work

The framework draws on linearizability and transaction-processing ideas [Herlihy and Wing, 1990, Gray and Reuter, 1993], causal ordering [Lamport, 1978], durable distributed-data design [Kleppmann, 2017], canonical serialization [Rundgren et al., 2020], and software supply-chain provenance [Torres-Arias et al., 2019, Open Source Security Foundation, 2023, Souppaya et al., 2022]. Smart-contract testing research contributes fuzzing, differential analysis, and large-scale tool comparison [Wüstholtz and Christakis, 2020, Durieux et al., 2020]. The novel emphasis here is not another testing tool. It is a claim architecture that binds financial authority, runtime identity, cross-layer derivation, reproducible packaging, and promotion language into one reviewable object.

AEMB v0.1.0 already treats protocol conformance as case-wise and conjunctive rather than compensating [Nechepurenko, 2026a]. AVET provides provenance-aware synthetic trace objects for venue emulation [Nechepurenko, 2026b]. The present system uses those ideas at the implementation-evidence layer.

D Publication and reproducibility record

The Phase 17B root topology is the common-cohort case: seven retained positions per FIA are published with registered derivation metadata, while the raw derivation payloads are intentionally absent from the public companion. Consequently, the observed result supports cohort consistency and registered provenance, not a claim that seven independent runtime systems observed each FIA. The proposed multi-root corroboration gate remains `NOT_PROVEN`.

The public evidence companion retains the exact source commit, manifest, ledger, materialization, package, replay, and binding identities used in this study, together with the per-FIA correlation records, schemas, checksums, and an offline metadata verifier. Two clean materializations and the separately verified strict archive provide the recorded package-level reproducibility identities; the selected P01–P17 chain provides the historical replay identity. Raw runtime payloads are intentionally excluded, so the publication verifier checks the integrity and consistency of published metadata rather than reconstructing the unpublished raw roots.

The public article reports the evidence method and registered identities. Broader source-code publication, operational deployment, and successor external-operation experiments are governed by

separate release and assurance processes.

E Limitations

The framework cannot prove that an oracle is truthful, a venue is solvent, a key is uncompromised, a smart contract has no unknown vulnerability, or a legal claim will be enforced. Multi-root corroboration is a practical, claim-specific property, not metaphysical independence; shared code, libraries, operators, or infrastructure can create common-mode error. Hashes detect changed bytes under specified assumptions, not semantic omissions outside the schema. The current evidence establishes the registered cohort and provenance predicates. Production performance, real custody, external-venue behavior, terminal settlement, and human usability remain separate empirical questions.

F Conclusion

End-to-end evidence for financial protocols should be treated as a typed, manifest-bound, reproducible proof chain. A receipt, screenshot, test, or copied hash can be useful without proving the whole workflow. The architecture developed here requires separate retained derivations, a shared correlation identity, specified tamper rejection, clean-run reproducibility, historical provenance, and independent review before a claim is promoted. Its most important output is a disciplined, reproducible boundary between what was observed, what was reconstructed, what was replayed, and what remains a future hypothesis.

References

- AxientLab. Axient FIA Public: Reproducibility and Evidence Companion. Public GitHub repository, 2026. URL <https://github.com/AxientLab/axient-fia-public>. Pinned commit 7722146bee0e7f02d0c4f9f63ada6d527e6ff079; accessed 12 August 2026.
- Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279, 2000. doi: 10.1145/351240.351266. URL <https://doi.org/10.1145/351240.351266>.
- Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the 42nd International Conference on Software Engineering*, pages 530–541, 2020. doi: 10.1145/3377811.3380364. URL <https://doi.org/10.1145/3377811.3380364>.
- Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.
- Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. doi: 10.1145/78969.78972. URL <https://doi.org/10.1145/78969.78972>.
- Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62. URL <https://doi.org/10.1109/TSE.2010.62>.
- Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. doi: 10.1145/359545.359563. URL <https://doi.org/10.1145/359545.359563>.

- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- Maksym Nechepurenko. AEMB: A Deterministic Cross-Language Verification Benchmark for Event-Margin Protocols. *SSRN Electronic Journal*, July 2026a. doi: 10.2139/ssrn.7216198. URL <https://doi.org/10.2139/ssrn.7216198>. SSRN abstract 7216198; 22 July 2026; AEMB v0.1.0.
- Maksym Nechepurenko. AVET: A Provenance-Aware Venue Emulator Trace Dataset for Event Markets. *SSRN Electronic Journal*, July 2026b. doi: 10.2139/ssrn.7216180. URL <https://doi.org/10.2139/ssrn.7216180>. SSRN abstract 7216180; 27 July 2026; AVET v0.2.0.
- Open Source Security Foundation. Supply-chain Levels for Software Artifacts (SLSA) Specification, Version 1.0, 2023. URL <https://slsa.dev/spec/v1.0/>. Final specification, 19 April 2023; accessed 12 August 2026.
- Anders Rundgren, Bryce S. Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). Technical Report RFC 8785, Internet Engineering Task Force, 2020. URL <https://doi.org/10.17487/RFC8785>.
- Murugiah Souppaya, Karen Scarfone, and Donna Dodson. Secure Software Development Framework (SSDF) Version 1.1. Technical Report NIST SP 800-218, National Institute of Standards and Technology, 2022. URL <https://doi.org/10.6028/NIST.SP.800-218>.
- Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>. Accessed 12 August 2026.
- Valentin Wüstholtz and Maria Christakis. Harvey: A greybox fuzzer for smart contracts. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1398–1409, 2020. doi: 10.1145/3368089.3417064. URL <https://doi.org/10.1145/3368089.3417064>.